Inteligencia artificial para proyectos de infraestructura ASIR

Unidad 6 · Proyecto Intermodular II

Paco Aldarias

7 de noviembre de 2026

Índice

Inteligencia artificial para proyectos de infraestructura ASIR **1**
1. Tipos de inteligencia artificial . . . 1
Según su capacidad . . . 2
Según cómo aprende o resuelve tareas . . . 2
2. Herramientas de IA y usos en ASIR . . . 2
Ejemplos de tareas adecuadas . . . 3
Límites y uso responsable . . . 3
3. Qué es el *vibecoding* . . . 3
4. Etapas: de la idea a GitHub . . . 3
1. Definir el problema y el alcance . . . 3
2. Recopilar requisitos y restricciones . . . 3
3. Diseñar antes de generar . . . 3
4. Preparar el espacio de trabajo . . . 3
5. Dar contexto e instrucciones concretas . . . 4
6. Construir en incrementos pequeños . . . 4
7. Validar funcionalidad y seguridad . . . 4
8. Documentar y preparar la publicación . . . 4
9. Publicar y mantener en GitHub . . . 4
5. Integrar varias IA en Visual Studio Code . . . 4
6. GitHub y la IA desde VS Code . . . 5
7. Lista de comprobación para el proyecto . . . 5
8. Fuentes para ampliar . . . 5

Volver al índice principal

Inteligencia artificial para proyectos de infraestructura ASIR

La inteligencia artificial (IA) reúne técnicas que permiten a los sistemas realizar tareas como reconocer patrones, generar contenido, clasificar información o proponer respuestas. En un proyecto de Administración de Sistemas Informáticos en Red (ASIR), puede ayudar a investigar, diseñar, documentar y automatizar; no sustituye la comprensión técnica ni las pruebas.

> **Idea clave:** la IA propone; el equipo del proyecto revisa, prueba y asume la responsabilidad de lo que instala, configura y publica.

1. Tipos de inteligencia artificial

No existe una única clasificación: se puede hablar de IA según su capacidad, su método de aprendizaje o el tipo de tarea que realiza. Las categorías siguientes se solapan.

Según su capacidad

- **IA estrecha o especializada (ANI):** está diseñada para tareas concretas, como traducir, recomendar contenido, detectar anomalías o generar código. Es el tipo de IA que usamos hoy.
- **IA general (AGI):** sería capaz de aprender y resolver de forma flexible una gran variedad de tareas intelectuales, con una competencia comparable a la humana. No existe hoy como tecnología demostrada y disponible.
- **Superinteligencia artificial (ASI):** hipotética IA que superaría ampliamente las capacidades humanas en numerosos campos. Es una idea teórica, no un producto actual.

Según cómo aprende o resuelve tareas

- **IA simbólica o basada en reglas:** aplica reglas explícitas, lógica y conocimiento introducido por personas. Puede resultar comprensible y predecible, pero depende de que las reglas cubran los casos relevantes.
- **Aprendizaje automático (machine learning):** aprende patrones a partir de ejemplos. Puede ser supervisado (con ejemplos etiquetados), no supervisado (busca estructuras sin etiquetas) o por refuerzo (aprende mediante acciones y recompensas).
- **Aprendizaje profundo (deep learning):** utiliza redes neuronales con muchas capas para aprender patrones complejos, por ejemplo en imágenes, voz y lenguaje. Es una familia de técnicas de aprendizaje automático.
- **IA generativa:** crea contenido nuevo —texto, imágenes, audio o código— a partir de una petición y del contexto. Los modelos de lenguaje grandes (LLM) son un ejemplo. Pueden producir respuestas convincentes pero incorrectas; no son una fuente de verdad por sí solos.

2. Herramientas de IA y usos en ASIR

Las herramientas cambian con rapidez y sus funciones dependen de la versión, el plan, la configuración y las políticas de cada organización. Estos son tipos de herramientas y ejemplos conocidos; conviene comprobar sus condiciones antes de usarlos.

Tipo de herramienta	Ejemplos	Posibles usos en un proyecto ASIR
Asistentes conversacionales	ChatGPT, Claude, Gemini	Aclarar conceptos, comparar alternativas, preparar listas de requisitos y revisar documentación.
Asistentes de programación	GitHub Copilot, asistentes de otros proveedores	Explicar código, proponer scripts, completar funciones, generar pruebas y ayudar a entender errores dentro del editor.
Generación de imágenes y diagramas	Generadores de imágenes y herramientas de diagramación con IA	Crear una ilustración conceptual o un primer borrador de una arquitectura. Verificar que la topología y las etiquetas sean técnicamente correctas.
Herramientas de búsqueda y síntesis	Buscadores con respuestas asistidas por IA	Localizar documentación y obtener un resumen inicial. Contrastar las respuestas con documentación oficial y fecha de publicación.
Modelos ejecutados localmente	Herramientas como Ollama, con modelos compatibles	Experimentar sin enviar cada petición a un servicio remoto. El resultado depende del modelo, el hardware y la configuración; “local” no garantiza por sí solo privacidad ni exactitud.
Automatización y operaciones	Funciones inteligentes incluidas en plataformas de monitorización, nube o gestión	Resumir alertas o señalar patrones. Revisar siempre el diagnóstico antes de modificar sistemas.

Ejemplos de tareas adecuadas

- Pedir una explicación de las diferencias entre DNS recursivo y autoritativo, y verificarla en documentación fiable.
- Solicitar un borrador de inventario de servidores, una lista de requisitos o un esquema de secciones para la memoria técnica.
- Pedir que explique un script de Bash o PowerShell, o que sugiera casos de prueba para un script propio.
- Obtener un primer borrador de una configuración de laboratorio, para después validarlo en un entorno aislado.
- Resumir mensajes de error anonimizados y recibir hipótesis ordenadas para investigar.

Límites y uso responsable

- La IA puede inventar parámetros, comandos, opciones de software o referencias. Comprueba los detalles en documentación oficial.
- No pegues contraseñas, claves SSH, tokens, datos personales, direcciones sensibles ni configuraciones confidenciales en servicios externos.
- Elimina o sustituye nombres de host, dominios internos, direcciones IP públicas y cualquier dato que identifique una organización antes de compartir registros.
- No ejecutes comandos sugeridos directamente en producción. Léelos, entiende sus efectos y pruébalos en una máquina virtual o laboratorio.
- Revisa los permisos, las licencias y las políticas del centro o de la empresa para el código y los datos usados.
- Mantén evidencias de las pruebas y documenta qué partes se generaron o revisaron con IA, siguiendo las instrucciones del proyecto.

3. Qué es el *vibecoding*

Vibecoding (también escrito *vibe coding*) es una forma informal de crear software describiendo en lenguaje natural lo que se quiere y usando una IA para generar o modificar código mediante iteraciones. La persona dirige el trabajo con instrucciones y comentarios, mientras la IA propone cambios.

No significa que el proyecto se construya sin diseño ni revisión. En infraestructura, un fragmento incorrecto puede interrumpir servicios, abrir un acceso o borrar datos. Por eso, el *vibecoding* debe tratarse como una ayuda de desarrollo, no como una autorización para delegar decisiones o ejecutar cambios a ciegas.

4. Etapas: de la idea a GitHub

1. Definir el problema y el alcance

Describe qué necesidad resuelve el proyecto, quién lo usará y qué queda fuera. Por ejemplo: “montar un laboratorio reproducible con DNS, DHCP y monitorización para una red de pruebas”. Evita empezar con una petición vaga como “crea toda mi infraestructura”.

2. Recopilar requisitos y restricciones

Anota requisitos funcionales y no funcionales: servicios, usuarios, seguridad, disponibilidad, presupuesto, recursos, plazos y criterios de aceptación. Separa los datos confirmados de las suposiciones y no compartas secretos con el asistente.

3. Diseñar antes de generar

Decide la arquitectura y el entorno de pruebas: máquinas virtuales, contenedores o nube; topología; direcciones reservadas para el laboratorio; dependencias; copias de seguridad y controles de acceso. Pide a la IA alternativas y riesgos, pero justifica tú la decisión final.

4. Preparar el espacio de trabajo

Crea una carpeta y un repositorio Git. Añade un `README.md`, organiza los directorios y define cómo se probará el proyecto. Incluye un `.gitignore` adecuado para evitar publicar archivos temporales, secretos o estado local.

5. Dar contexto e instrucciones concretas

Indica al asistente el objetivo, el sistema operativo, las versiones relevantes, las restricciones y el resultado esperado. Pide cambios pequeños, archivos concretos y una explicación de cómo validarlos. No incluyas credenciales ni datos reales.

6. Construir en incrementos pequeños

Implementa una parte cada vez: primero una configuración mínima, después una función o servicio, y luego la siguiente integración. Tras cada paso, inspecciona el diff de Git, lee el código y ejecuta las pruebas en el laboratorio.

7. Validar funcionalidad y seguridad

Comprueba sintaxis, funcionamiento, permisos, puertos expuestos, manejo de errores, registros y recuperación. Usa validadores propios del formato cuando existan (por ejemplo, `terraform validate` para Terraform) y realiza pruebas de conectividad y de aceptación. No des por válida una respuesta solo porque "parece funcionar".

8. Documentar y preparar la publicación

Explica la arquitectura, los requisitos, la instalación, la configuración, las pruebas y la recuperación ante fallos. Sustituye secretos por variables de entorno o ejemplos ficticios. Revisa el historial y los archivos para confirmar que no se ha incluido ninguna clave.

9. Publicar y mantener en GitHub

Revisa el estado del repositorio y los cambios, crea commits con mensajes claros y publícalos en un repositorio de GitHub con visibilidad apropiada. Añade instrucciones de uso y licencia si corresponde. Si el trabajo continúa, usa ramas y revisiones mediante *pull requests*. Tras publicar, comprueba desde la web que los archivos y la documentación se ven correctamente.

> **Antes de publicar:** busca tokens, contraseñas, claves privadas, archivos `.env`, copias de seguridad y datos personales. Si se publica una credencial por error, borrarla del último commit no basta: revócala y rótala.

5. Integrar varias IA en Visual Studio Code

Visual Studio Code (VS Code) puede reunir el editor, GitHub y asistentes de IA mediante extensiones. La integración exacta varía; algunas extensiones usan la cuenta de su proveedor, otras una suscripción, una clave configurada por el usuario o un modelo local. Instala extensiones de editores y proveedores en los que confíes y confirma sus condiciones de privacidad.

1. **Instala VS Code y Git.** Comprueba que Git está disponible y abre la carpeta del proyecto como espacio de trabajo.
2. **Instala el asistente de GitHub.** Busca la extensión oficial de GitHub Copilot en la vista de extensiones, revisa el editor y los permisos, instálala e inicia sesión si tienes acceso. Sus prestaciones dependen del plan y la configuración.
3. **Añade otros proveedores si los necesitas.** Busca sus extensiones oficiales o mantenidas por el proveedor en el Marketplace. Sigue su documentación para iniciar sesión, configurar el modelo o conectar un servicio local.
4. **Usa cada asistente para una tarea acotada.** Por ejemplo, uno puede ayudarte a explicar un error y otro a revisar la documentación. Evita enviar el mismo código privado a varios servicios sin autorización.
5. **Revisa las respuestas en el propio proyecto.** Las sugerencias no son cambios verificados: inspecciona los archivos modificados, consulta el diff y ejecuta las pruebas.
6. **Comprueba la confianza del espacio de trabajo.** No habilites extensiones ni ejecutes tareas de una carpeta desconocida sin revisar sus permisos y configuración.

Tener varias extensiones instaladas no implica que colaboren entre sí ni que compartan contexto automáticamente. Pueden duplicar sugerencias, requerir cuentas distintas o enviar datos a servicios diferentes. Empieza con una herramienta, evalúa su utilidad y añade otra solo si aporta valor.

6. GitHub y la IA desde VS Code

Son piezas relacionadas, pero distintas:

- **Git** registra versiones y cambios del proyecto en el equipo.
- **GitHub** aloja repositorios Git y permite colaborar, revisar cambios y gestionar incidencias.
- **La extensión GitHub** facilita iniciar sesión y trabajar con repositorios o funciones de GitHub desde VS Code.
- **GitHub Copilot** es un asistente de IA integrado mediante extensiones; no es lo mismo que Git ni que GitHub.

Flujo habitual: abre una carpeta, inicializa o clona el repositorio Git, crea y revisa cambios, confirma los cambios (*commit*), inicia sesión en GitHub y publica o sincroniza el repositorio (*push*). Desde el editor se puede usar un asistente para explicar o proponer código y, a la vez, utilizar el control de versiones para inspeccionar y guardar los cambios. Algunas funciones dependen de las extensiones y permisos instalados.

La IA no publica de forma responsable por sí sola: antes de enviar cambios, verifica que el repositorio no contenga secretos, información sensible ni configuraciones inseguras.

7. Lista de comprobación para el proyecto

- ☐ El problema, el alcance y los criterios de aceptación están escritos.
- ☐ Las decisiones de arquitectura se entienden y están justificadas.
- ☐ El código y las configuraciones sugeridas por IA se han revisado.
- ☐ Las pruebas se ejecutaron en un laboratorio aislado y sus resultados están documentados.
- ☐ No hay claves, datos personales ni información confidencial en los archivos o en el historial Git.
- ☐ El `README.md` explica cómo preparar, probar y detener el proyecto.
- ☐ El repositorio de GitHub tiene la visibilidad y los permisos adecuados.

8. Fuentes para ampliar

- Documentación de GitHub Copilot en VS Code
- Control de código fuente en VS Code
- Uso de Copilot en el IDE de GitHub
- Crear un repositorio en GitHub

Las interfaces, nombres de funciones y condiciones de los servicios pueden cambiar. Consulta la documentación oficial vigente y las normas de tu organización.

Descargar estos apuntes en PDF

Volver al índice principal